IST 387: Introduction to Applied Data Science
Lab 3: Part 1 Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Introduction
- Why create and use functions?
- Creating Functions in R
- Defensive Coding
- Installing a Package to Access a Function
- Case Study: Creating and Using a Calculate NPS Function
- Important R Functions

Preeti Jagadev

Introduction

We have already been using many functions, such as **mean(), hist(), min(),** and **max(),** that are provided with R. Those functions were written by the creators of R, and they are provided by default when R is installed, meaning that we don't have to show R how to compute the mean, min, or max with detailed code- we can simply call these functions instead. People who use R regularly often create their own new functions so that they can achieve that same level of convenience by automating a new process or a calculation.

Why create and use functions?

The two most important reasons why people use functions are as follows:

- **Reusability:** Once a function is defined, it can be used repeatedly. We can invoke the same function many times in our program, and this saves us work.

- **Abstraction:** After a function has been created, we don't need to remember the details of how it works to use it.

Once a function has been created and is known to R, we only need to know the following four things:

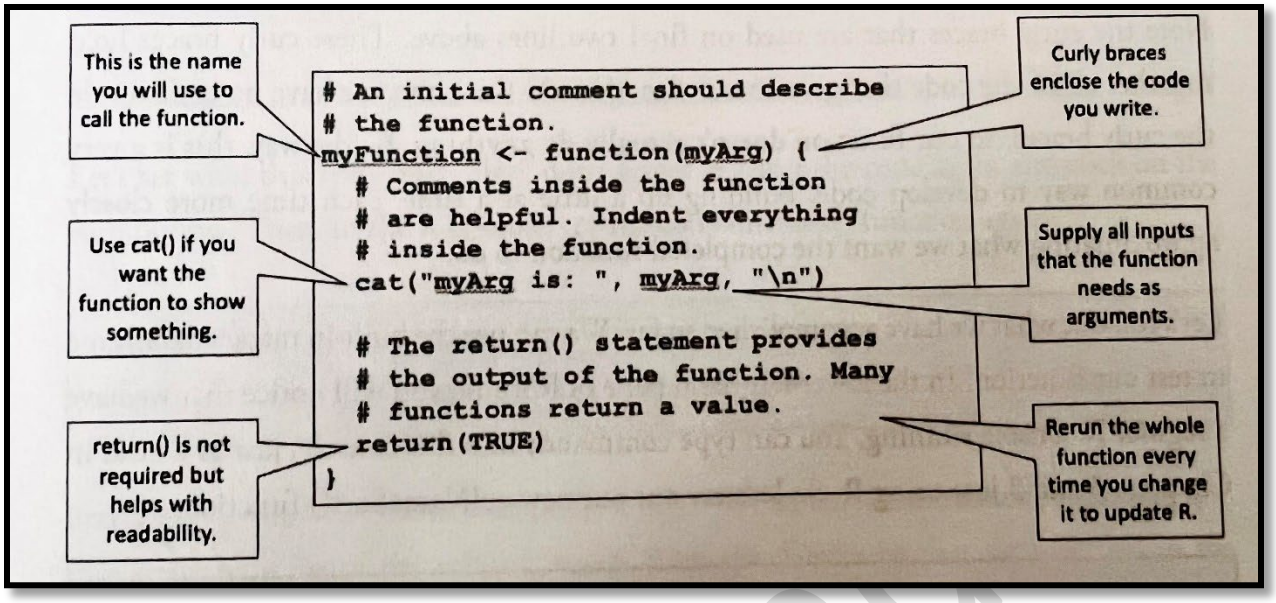
1. The name of the function.
2. Its purpose (i.e., Why, and when we would want to use it.)
3. What arguments must be supplied to the function; and
4. What kind of result the function returns.

Argument: Argument (also known as parameter) is a term used by computer scientists to refer to some information that is sent into a function to help it do its job. For **mean( ),** we pass in one argument, the vector on which we should calculate the mean. Other functions take more than one argument. One such function is **head( )**, which can take two arguments. The second argument to **head( )** is optional**,** which means that R only needs it in certain circumstances and/or R can supply a default value if the user does not pass in a value.

Creating Functions in R

Most functions in R have a common set of components: Name, some optional arguments, some code to run, an optional return value. The figure below shows a simple example of a function and points out its key components.

Preeti Jagadev



The figure above, illustrates the features a function must have.

- The name of this function is "**myFunction**".
- The function has one argument, called "**myArg**", that we can use to supply some information to the function.
- The function does a little bit of work- specifically it contains a call to **cat()** that will print some information on the console.
- The function ends with a **return statement:** in this case we return the same thing every time, and that is the value TRUE.

Other than serving as an illustration, this is not a useful function, so let's build our own function to perform a simple but useful trick:

Alphabetically sort the column names of a dataframe

- ***Step 1:***

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){

}
```

You have just created your first function in R. We have called our function **colNameSort( ).** Our new function has just one argument, called **inputDF.** We have not said anything about what **inputDF** will contain and R is flexible about that.

Note the curly braces that are used on final two lines of the code above. These curly braces hold together all the code that goes in our function. Currently, we have no code inside the curly braces, so our function doesn't do anything.

Let's test what we have accomplished so far. We can use the **built-in mtcars dataframe** to test our function:

```
colNameSort(mtcars)
```

The output is as follows:

```
> colNameSort(mtcars)
NULL
```

We got a return value of NULL, which is R's way of saying that nothing was returned.

- ***Step 2:***

Now we ready to add some code to our function:

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){
  nameList <- colnames(inputDF)
  cat(nameList)
}
colNameSort(mtcars)
```

The output is as follows:

```
> colNameSort(mtcars)
mpg cyl disp hp drat wt qsec vs am gear carb
```

Pretty easy to see what the new code does. We called the **colnames( )** function, and that returned a list of the column names from the dataframe that we specified in the argument. Then we used **cat( )** to report that list of names to the console.

- ***Step 3:***

Now let's build a little more code.

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){
    nameList <- colnames(inputDF)
    nameList <- sort(nameList)
    cat(nameList)
}
colNameSort(mtcars)
```

In the above code we have extracted the column names from the dataframe supplied as the argument and sorted this list alphabetically before displaying it on the console. The output is as follows:

```
> colNameSort(mtcars)
am carb cyl disp drat gear hp mpg qsec vs wt
```

- ***Step 4:***

Let's now rearrange the columns in the dataframe, using the sorted list. Further, let's also use **cat()** to report the list of column names from our new copy of the dataframe, which we have named outputDF:

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){
    nameList <- colnames(inputDF)
    nameList <- sort(nameList)
    outputDF <- inputDF[nameList]
    cat(colnames(outputDF))
}
colNameSort(mtcars)
```

The output is as follows:

```
> colNameSort(mtcars)
am carb cyl disp drat gear hp mpg qsec vs wt
```

- ***Step 5:***

There's one last step that we need to take. We want to return the **outputDF** that we created:

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){
    nameList <- colnames(inputDF)
    nameList <- sort(nameList)
    outputDF <- inputDF[nameList]
    return(outputDF)
}
alphaMtcars <- colNameSort(mtcars)
str(alphaMtcars)
```

The output is as follows:

```
> str(alphaMtcars)
'data.frame':   32 obs. of  11 variables:
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
```

Perfect! The dataframe that our function returned has all the original columns from mtcars, but now they are in alphabetical order.

Defensive Coding

In the previous section we designed **colNameSort( )** to accept a dataframe as input and to return a dataframe as its output. What if we gave our function something it did not expect? Let's try it with the built in Titanic data, which is stored as a table rather than a dataframe:

```
colNameSort(Titanic)
```

The output is as follows:

```
> colNameSort(Titanic)
[1] NA NA
```

Our function returned two NA values. However, we got no warning or notification that the function failed to do its job. Let's add a feature to our function that checks to make sure that the incoming argument is a dataframe:

```
#create a function to sort a dataframe's columns
#alphabetically by column name
colNameSort <- function(inputDF){

  if (is.data.frame(inputDF)) {
    nameList <- colnames(inputDF)
    nameList <- sort(nameList)
    outputDF <- inputDF[nameList]
    return(outputDF)
  } else {
    warning("Expecting a dataframe as input.\n")
    return(NULL)
  }
}
colNameSort(Titanic)
```

We have added an **if-else statement** to this function. The **if statement** uses **is.data.frame( )** to test whether the input is what we expect. This returns **TRUE** if **inputDF** is a dataframe and **FALSE** if it is anything else. If **TRUE,** we proceed with the same code as before. The **else** statement handles the **FALSE** condition, and you can see that it generates a warning message explaining what happened and returns a NULL instead of an output dataframe. The output is as follows:

```
> colNameSort(Titanic)
NULL
Warning message:
In colNameSort(Titanic) : Expecting a dataframe as input.
```

In the codes above, we introduced new functions such as such as **warning( )** and **return( ).** Where did these come from and how did we know where to find them and how to use them? R has so many functions that it is difficult to memorize them all. Search online and you will find dozens of instances where people have probably tried to solve similar problems to the one you are solving, and you will also find they have posted the R code for their solutions. These code fragments are free to borrow and test. In fact, learning from other people's examples is a great way to expand your horizons and learn new techniques. In the next section let's look at how to install packages and find out about new functions.

Installing a Package to Access a Function

The **dplyr package** contains several very helpful functions for helping to manipulate data. Let's write another implementation of our **colNameSort()** code using the **select( )** function from the **dplyr package**.

To install this package, look in the lower-right-hand pane of RStudio. There are several tabs there, and one of them is **"Packages".** Click on this and you will get a list of every package that you already have available in your copy of R with checkmarks for the ones that are ready to use. You probably have the **dplyr package** already installed, as a side effect of installing the **tidyverse.** If not, then click on the install button at the top of the window and type dplyr in the dialog box. If not, then click on the Install button at the top of the window and type **dplyr** in the dialog box. There might be a checked box for **Install Dependencies**, and if so, leave this checked. In some cases, an R package will depend on other packages and R will install all the necessary packages in the correct order if it can. Once you click the Install button in this dialog, you will see some commands running on the R console. Once the installation is complete you will see **dplyr** added to the list in the lower-right pane (assuming you have clicked the Packages tab). One last step is to click the check box next to it. This runs the library function on the package, which prepares it for further use.

Preeti Jagadev

Let's look at the help for the **select function**. There are three ways to get the help:

- Type **help("select")** at the command line.
- Type **?select** at the command line, or
- Click on the name **dplyr** in the Packages window and navigate to the function you want using the topical index.

The help shows that the long title for the function is "***Select/rename variables by name***" and the brief description of the function is "***Choose or rename variables from a tbl.select() keeps only the variables you mention***." The help page goes on to describe the inputs and outputs of the function and gives some examples of how to use it.

Let's use the **select()** function together with **sort()** and **names()** to rebuild our **colNameSort()** function:

```
#To access the select() function we need dplyr
library(dplyr)

colNameSort <- function(inputDF) {

  inputDF %>% select(sort(names(.)))

}
colNameSort(mtcars)
```

- The first line, a call to **library()** is included when we are about to use a function from a new library. Here, with **library(dplyr),** We are making it possible to use the **select()** function.

- We have shrunk our function down to a single line of code by piping **inputDF** into our **select( )** expression. The innermost expression, **names( ),** returns a list of the column names from the data structure that was received from the pipe. The dot in that expression is just a shortcut for, "*that thing we just got*," and it can be used whenever needed within a pipe expression. So, when we run **sort (names(.))** we get a nice alphabetical list of the column names from the **inputDF**.

- We are supplying that list to **select( )** so that it will know what columns to keep and the order in which to present them.

- Finally, we are taking advantage of a handy feature of R functions, which is that, without a call to **return( ),** the output value of the function is whatever the last expression in the function produced.

Function %>% in R

R pipes are a way to chain multiple operations together in a concise and expressive way. They are represented by the %>% operator, which takes the output of the expression on its left and passes it as the first argument to the function on its right. Using pipes in R allows us to link a sequence of analysis steps.

Case Study: Creating and Using a Calculate NPS Function

NPS is a measurement that many businesses use to quickly summarize the attitudes of a set of consumers towards the product or service that the business offers. To calculate NPS, we need a vector of numbers where each number represents a **likelihood to recommend,** wherein the question is "*On a scale of 1 to 10, how likely are you to recommend this product or service?* Below is the code that creates a function to calculate NPS:

```
#Calculate NPS
#ltr: A vector of likelihood-to-recommend values
#neutral: The range below which someone is considered
#a detractor and above which someone is considered
#a promoter (default is 7 to 8)

CalcNPS <- function(ltr, neutral=c(7,8)){

#Create a new vector with just the promoters
#then calculate the length of the promoters vector
  promoters <- ltr[ltr > max(neutral)]
  numPromoters <- length(promoters)

#Calculate the number of detractors
  detractors <- ltr[ltr < min(neutral)]
  numDetractors <- length(detractors)

#Calculate NPS
  total <- length(ltr)
  nps <- (numPromoters/total - numDetractors/total)*100
  return(nps)
}
ltr <- c(9,8,3,9,7,8,9,6,7,8,9)
CalcNPS(ltr)
```

The output is as follows:

```
> CalcNPS(ltr)
[1] 18.18182
```

One of the things you may have noticed about this function is that it takes two arguments. The second argument called **neutral** can be used to specify the neutral zone where a respondent is considered neither a promoter nor a detractor. When we mention **neutral**, we also use an equal sign and provide the default range of 7 to 8.

Consider the example given below, which involves the **smallSurvey.csv**, which consists of the data that was used to calculate the NPS value:

```
library(readr)
smallSurvey <- read_csv("smallSurvey.csv")

#Remove the NAs from LTR column
smallSurvey <- filter(smallSurvey, !is.na(LTR))

#The mean likelihood to recommend by Type of travel.
smallSurvey %>% group_by(Type.of.Travel) %>% summarize(mean=mean(LTR), count=n())
```

The output is as follows:

```
# A tibble: 4 × 3
  Type.of.Travel   mean count
  <chr>           <dbl> <int>
1 Business travel  8.14   303
2 Mileage tickets  8.05    39
3 Personal Travel  5.46   156
4 NA               7        1
```

In the code above, we read the survey from the CSV file, filter out all the rows that have missing data for LTR and then calculate the mean for each type of travel. The output shows that there is one row that has a missing code for type of travel, but we can ignore that for now. Based on this analysis, we can see that, collectively people who do personal travel have lower LTR, as compared to business and mileage rewards travelers.

Next, let's calculate NPS, first for all the respondents and then broken down by group:

```
CalcNPS(smallSurvey$LTR)
```

The output is as follows:

```
> CalcNPS(smallSurvey$LTR)
[1] 9.018036
```

Further let's try the following command:

```
smallSurvey %>% group_by(Type.of.Travel) %>% summarise(nps=CalcNPS(LTR), count=n())
```

The output is as follows:

```
# A tibble: 4 × 3
  Type.of.Travel    nps count
  <chr>           <dbl> <int>
1 Business travel  40.3   303
2 Mileage tickets  30.8    39
3 Personal Travel -57.1   156
4 NA                0       1
```

These results show that NPS is highest for business travelers and lowest for personal travelers. In fact, the value for personal travelers is negative, indicating that there are more detractors than promoters in this group.

Important R Functions

- **cat( ):** Concatenate and print information to the console (or a file).
- **colnames( ):** Displays a list of the column names in a dataframe.
- **filter( ):** Selectively returns rows from a dataframe based on a criterion (dplyr package).
- **function( ):** Creates a new function.
- **group_by( ):** Divides a dataframe into groups (dplyr package).
- **is.data.frame( ):** Tests whether a data object is a dataframe.
- **n( ):** Shows the number of observations in a group (dplyr package).
- **names( ):** Produces a list of names of the elements of a structure such as a dataframe.
- **return( ):** Completes a function by returning a value.
- **select( ):** Chooses a subset of a dataframe (dplyr package).
- **sort( ):** Sorts the input vector alphabetically or numerically.
- **str( ):** Shows the structure of a data object.
- **sum( ):** Calculates the total of a numeric vector.
- **summarize( ):** Produces summary statistics for groups (dplyr package).
- **warning( ):** Produces a warning message on the console.

Preeti Jagadev